COMP1001 (Maths I) Exam Viewing

- I forgot to allocate 4 hours of slots for Maths exam viewing
- I will make these hours available, for 10 minute slots:
  - 3-5pm Thursday 12th March (1st hour you are free, 2nd hour is the Haskell lab)
  - 3-4pm Thursday 19th March (you are free, before the labs)
  - 2-3pm Friday 20th March (my office hour, which is usually quiet)

- Drop me an email with the subject of "Exam Viewing" and tell me: **your student id** (so I can find the exam) and **your preferred hour** (from the above) and I'll give you a slot within that hour, on first-come-first-served basis
  - Or offer alternatives if no space left in that hour

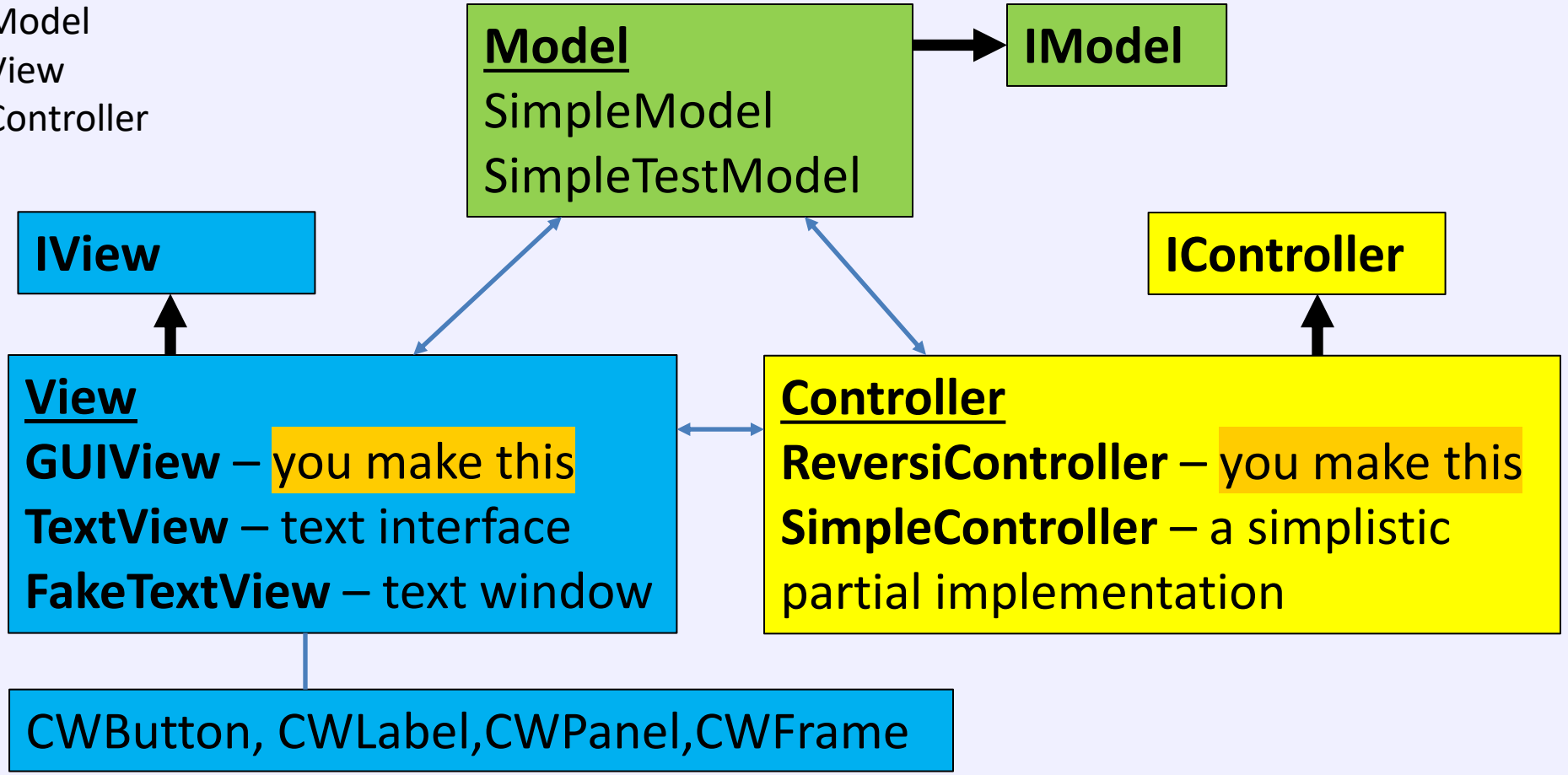
- If you have questions about Q2 or Q3b (Tim's lectures), then you need to book with him, but I can discuss Q1 and Q3a

COMP1009
Programming Paradigms

Lecture OO-10
Patterns &
Inner Classes

Model-View-Controller in CW2C&D

MVC pattern
Model
View
Controller



In ReversiMain, you create one object of each type, then call initialise on each to link them. You treat them as an IModel, IView, IController interface object.

This lecture

- Final
- Reviewing containers and components in Swing
- Inner Classes
  - And event handling

final

The use of ‘final’

- In Java final just means unchangeable/immutable
- Variables can be final: you can’t change the value once you have initialised them
  - Like ‘const’ in C/C++ (I have no idea why they renamed it)

```
public static final int MAX_VALUE = 3;
```

- Methods can be final: you can’t change their implementation in a sub-class (note: this can make it faster – no need to check which version to run)

```
final void myFunction() { … }
```

- Classes can be final: you can’t subclass/extend them

```
public final class FinalClass { … }
```

Final example

- An example using all of these in one class – experiment with these

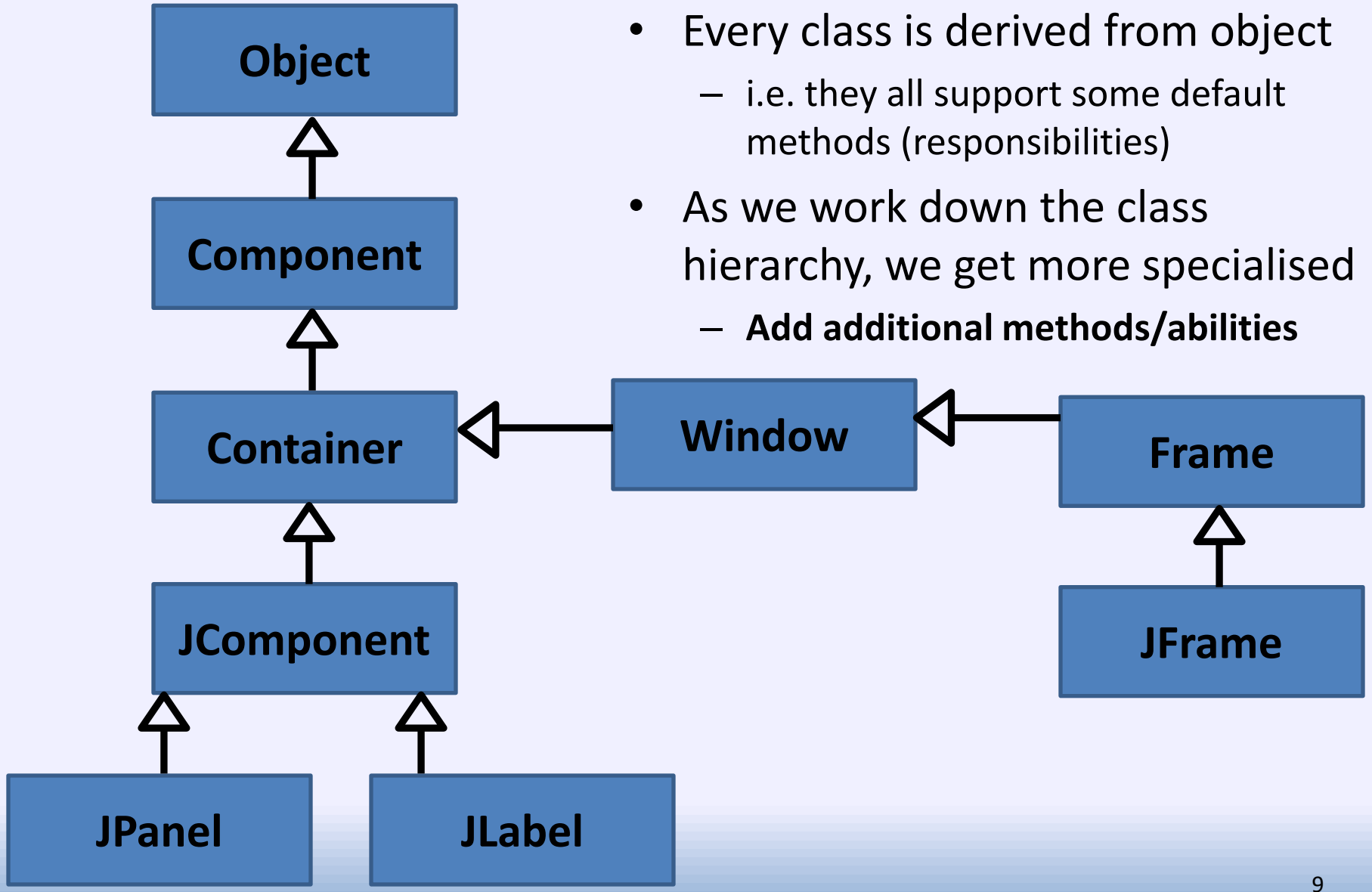
```
public final class FinalClass // Can't extend!
{
   public static final int MAX_VALUE = 3; // Constant

   public final void foo() // Can't override
   {
      System.out.println("FinalClass.foo()");
   }

   public int bar()
   {
      System.out.println("FinalClass.bar()");
      return 1;
   }
}
```

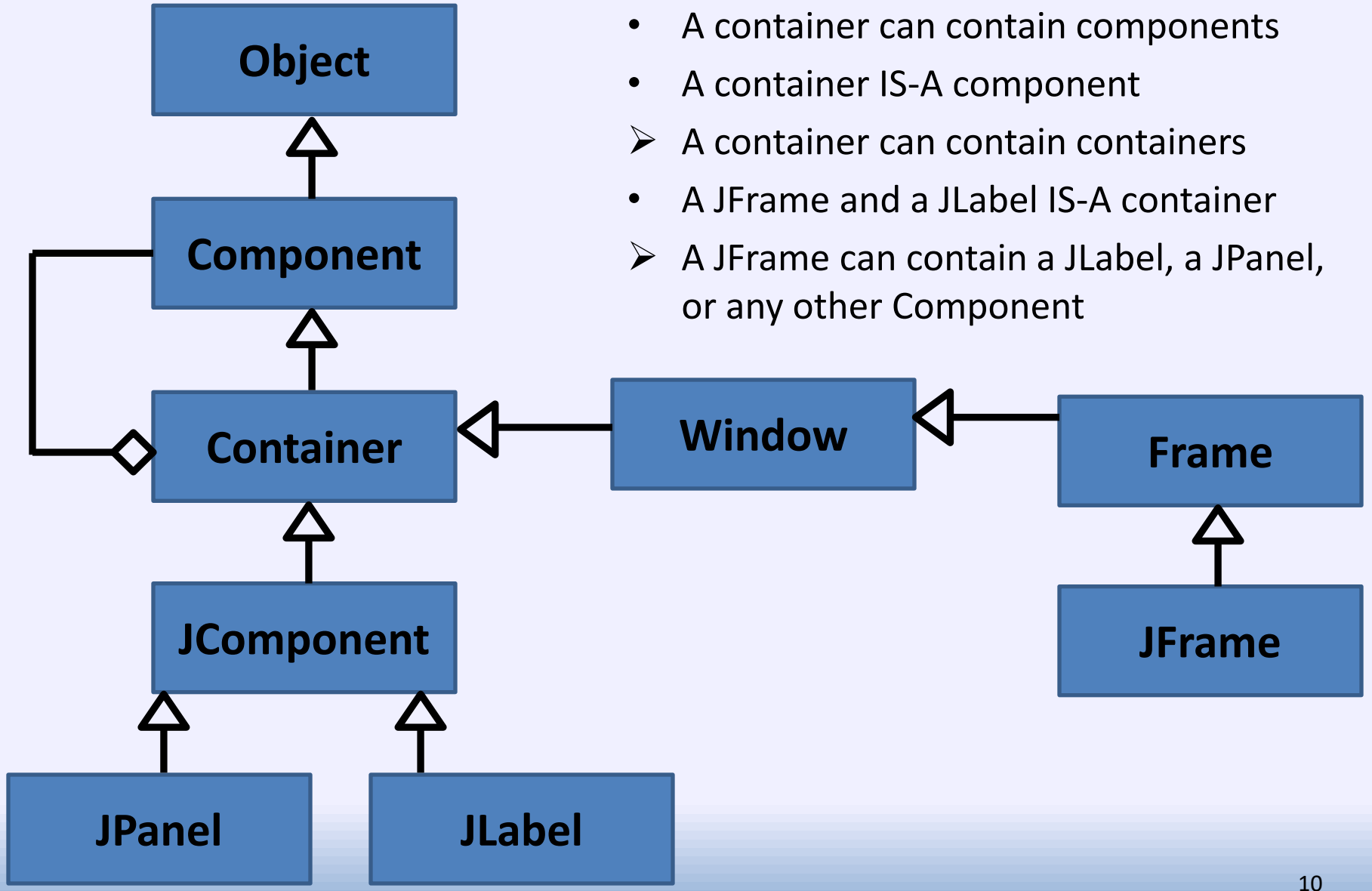
Reviewing our classes so far

The User Interface classes



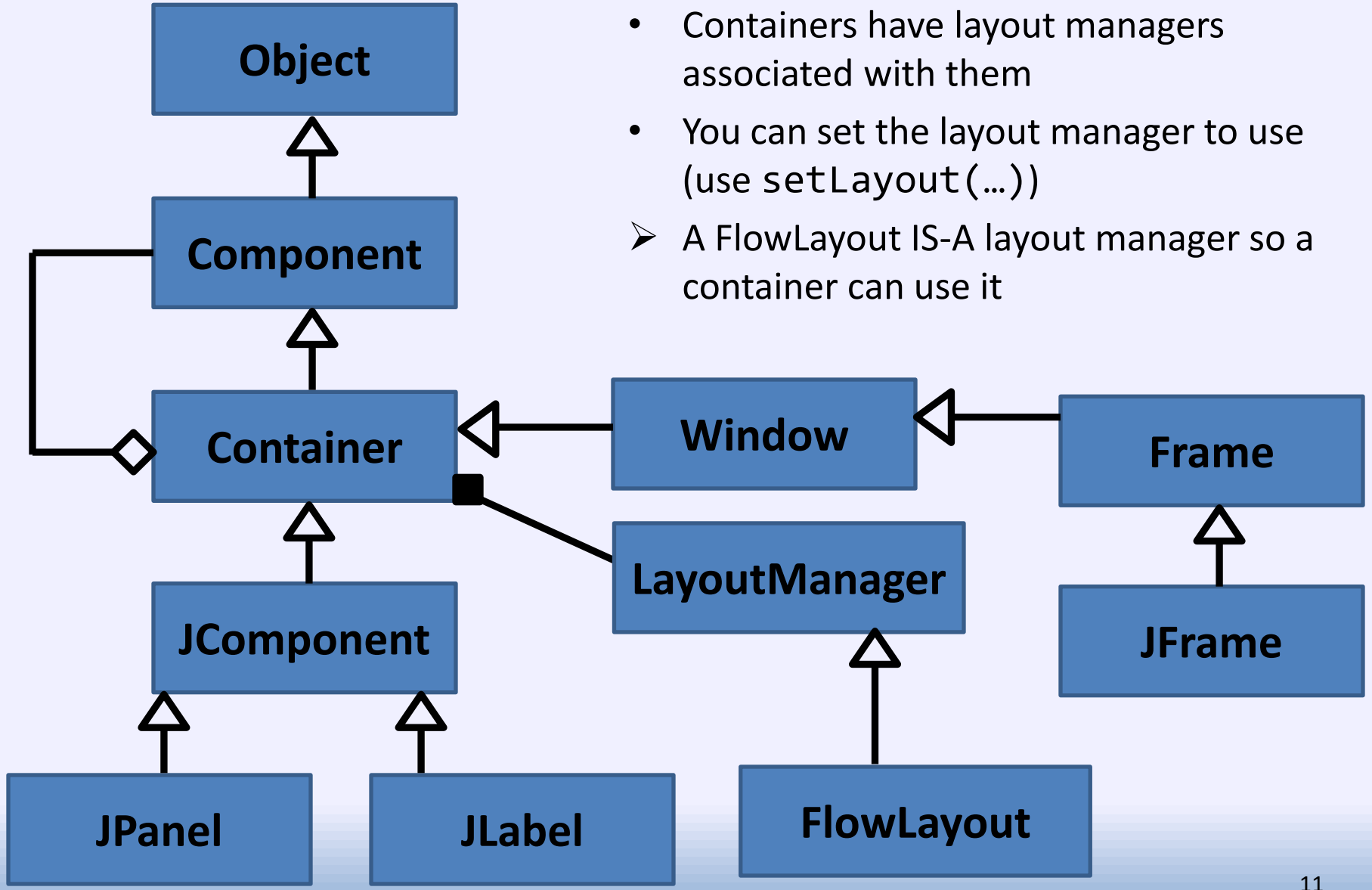
- Every class is derived from object
  - i.e. they all support some default methods (responsibilities)
- As we work down the class hierarchy, we get more specialised
  - **Add additional methods/abilities**

Containers HAVE components



- A container can contain components
- A container IS-A component
- ➢ A container can contain containers
- A JFrame and a JLabel IS-A container
- ➢ A JFrame can contain a JLabel, a JPanel, or any other Component

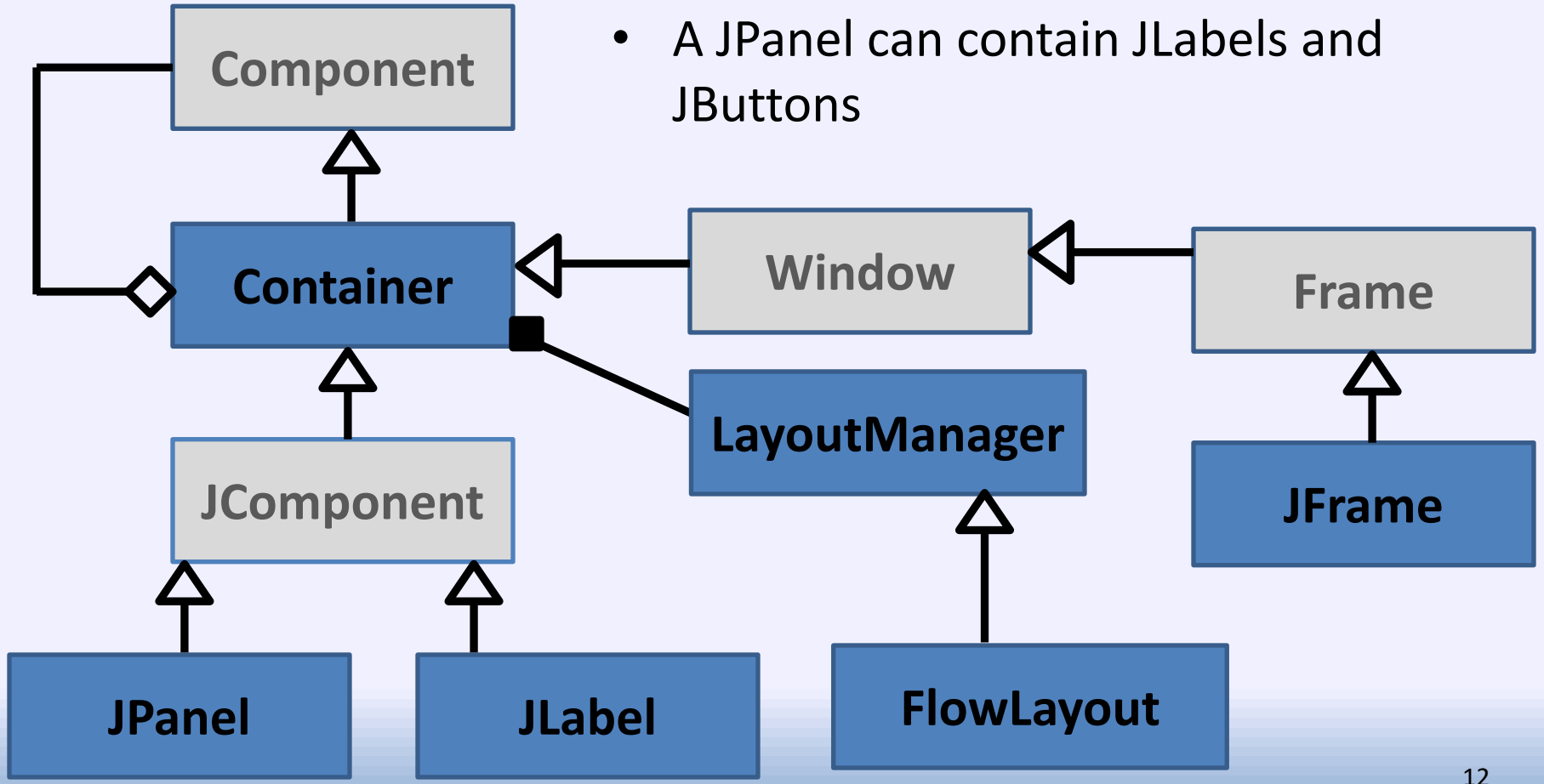
Layout managers



- Containers have layout managers associated with them
- You can set the layout manager to use (use `setLayout(…)`)
  - A FlowLayout IS-A layout manager so a container can use it

Classes we care about at the moment…

- We care that…
- A JFrame can contain JPanels and JLabels
- A JPanel can contain JLabels and JButtons

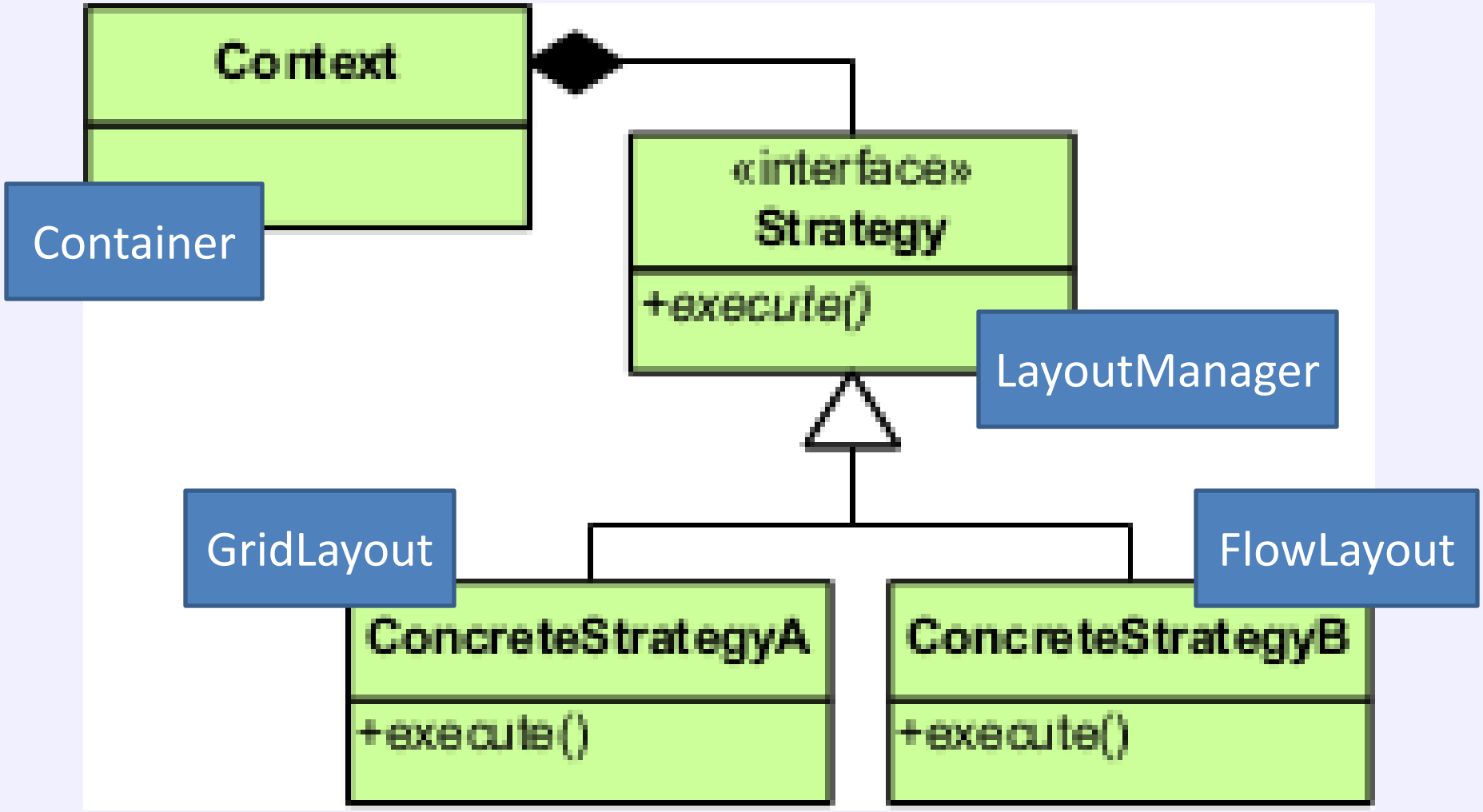


Patterns

We have seen two patterns so far

Both rely on sub-type inheritance

Strategy Pattern (Behavioural)

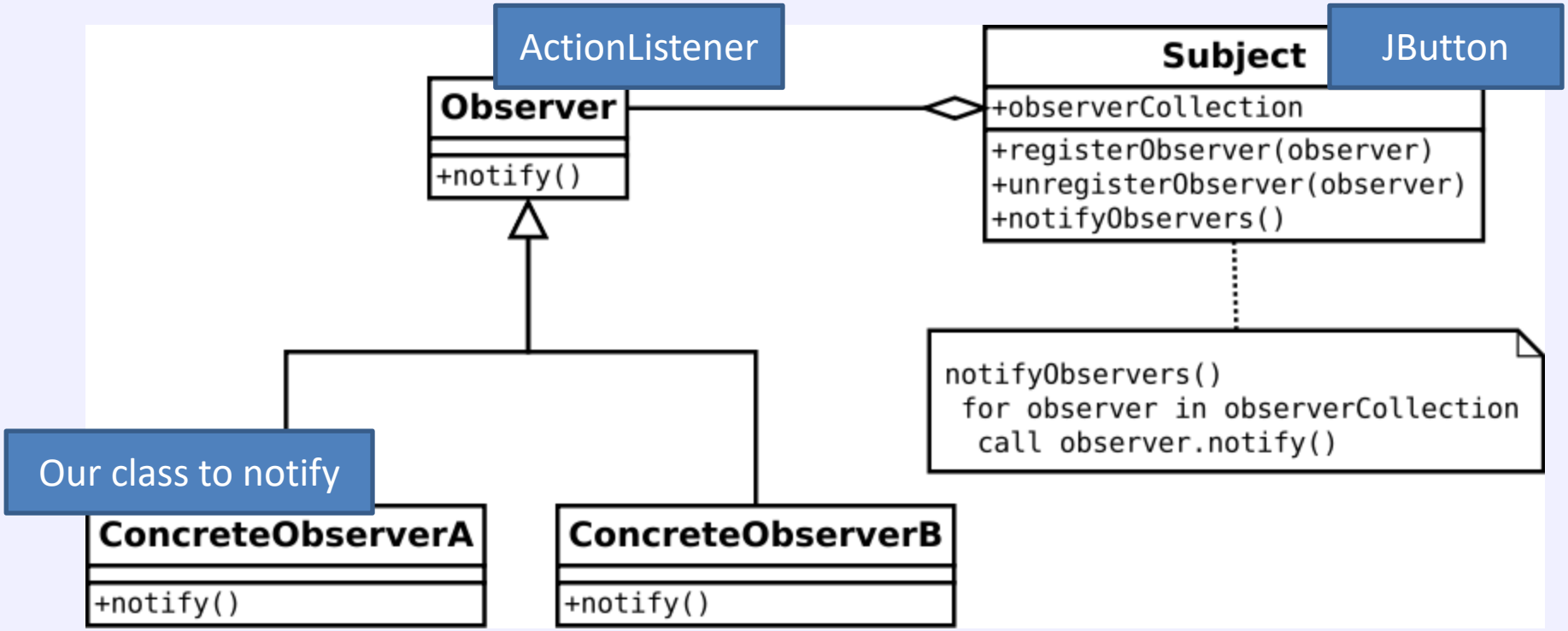


- Delegating some work to another class

Strategy Pattern Summary

- **Strategy Pattern is an example of a behavioural design pattern**
- Used when a class has multiple responsibilities and you want to **delegate some responsibility to another class**
  - Increases flexibility, simplifies each class
  - You can do this for multiple responsibilities
- For each responsibility to delegate, create an interface with the necessary functions to implement the functionality/strategy
  - E.g. how to lay out components, determine how much space is needed for components, etc
- Implementer objects (strategy objects) implement the functionality – in different ways – having different implementation of the functions
- Main object (context object) has an object reference of this **interface strategy** type (e.g. a LayoutManager reference), pointing to **ONE** object which will implement the functionality when needed
- Whenever this functionality/responsibility is needed, main object asks this single strategy object to do the action, rather than doing it itself
  - Uses the object reference to call the relevant functions, which calls the functions on the implementing class, which can do it in different ways

Observer pattern (Behavioural)



- Notifying an object (or multiple objects) of a different class that an event has occurred
  - Object to notify supports some interface (Observer here)
  - Notifying object keeps a list/array/etc of the objects to notify
  - Notifying object considers these as the base class/interface and calls a method

Observer Pattern Summary

- **Observer Pattern is an example of a behavioural design pattern**
- Used when a class wants to enable another object to get notifications about something (not to satisfy 'needs' of the subject object)
  - Objects have to 'register their interest in knowing about the event'
  - Zero of more objects may be registered at any time
- For each event type to know about, create an interface with the necessary function to call when the event happens
  - E.g. button was pressed, mouse moved, etc
- Thing to notify implements an observer object which implements the interface (and function(s)) – in different ways – having a different implementation of the functions
- Observer registers the object with the subject ('tell this object')
  - E.g. `subject.register( this );`
- Subject has a list of objects (interfaces) to notify, and adds the new one
  - E.g.
```
ArrayList<Observer> notifyList;
notifyList.add( newObserver);
```
- Subject treats the list contents as the interface type, calls the 'notify' function

```
for( Observer ob : notifyList )  // for each entry in list…
    ob.notify();                 // notify() it
```

Observer Pattern Example: ActionListener

- Button has a list of ActionListener objects to inform when it is pressed
- Something that wants to know creates an object that implements ActionListener, and gives the object reference to the Button

```
public interface ActionListener …
{
    public void actionPerformed(ActionEvent e);
}
public MyClass implements ActionListener …
```

- Things that want to know about the event tell it this by registering with it an object reference to the object which supports the interface
- Button stores the object reference it is given in its list of ActionListeners

- When a button is pressed, it will go through its list of registered ActionListeners, and call actionPerformed() on each
  - The actionPerformed() method of the object that implemented the interface, and registered itself with the button, will have its function called.

Compare and contrast

Strategy Pattern

- Example: LayoutManagers
- Purpose: A class wants to offer different ways to implement responsibility
  - E.g. different layout
- Exactly one strategy object for each strategy

Observer Pattern

- Example: ActionListeners on buttons
- Purpose: something wants to be notified when an 'event' happens
  - E.g. button press
- Zero or more observers for each subject

Both involve storing object reference(s) to other objects (one or a set), and calling functions on the objects using the object references

Nested and Inner Classes

Java classes

- So far, I mentioned that a class should be public and must go into a file with the same name as the class
  - And a directory the same as the package
- You can also define classes inside classes
  - This has specific meanings for these classes
  - Useful when a class is only used by the surrounding class
- You can make these classes:
  static (**not** associated with object of outer class)
  or not (is associated with an object of outer class)
  - Again static means 'not associated with an object'

An example of nested classes

```
public class Outer
{
   int i = 1;

   public static class NestedStatic
   {
      public void foo() {}
   }

   public class InnerClass
   {
      public void bar() { i = 2; }
   }
}
```

Nested class:
Defined within another class
static: Not associated with an Outer Class object

Inner class:
A nested class which is NOT static
IS associated with an Outer Class object

Inner class can access members of surrounding object

Creating instances of these classes

```
public class Outer {
   int i = 1;
   public static class NestedStatic …
   public class InnerClass…
   …
```

```
public class TestMain
{
   public static void main(String[] args)
   {
      Outer.NestedStatic ob2 = new Outer.NestedStatic();
      Outer ob1 = new Outer(); // Create object
      Outer.InnerClass ob3 = ob1.new InnerClass();
      ob2.foo();
      ob3.bar();
   }
}
```

Don't need to specify an object if it is obvious e.g. if a method of the outer class creates one

Using inner classes

A class just to start the program…

```
public class Startup
{
	public static void main(String[] args)
	{
		new MyApplication1().createGUI();
	}
}
```

- Create an instance of the MyApplication1 class and ask it to create its own GUI

Initial MyApplication…

```
public class MyApplication1
{
    JFrame guiFrame = new JFrame();
    ColorLabel labels[] = new ColorLabel[3];
    JButton but1; // Initialised to null as a member

    public void createGUI()
    {
        labels[0] = new ColorLabel( 100, 100, Color.RED );
        labels[1] = new ColorLabel( 100, 100, Color.BLUE );
        labels[2] = new ColorLabel( 100, 100, Color.GREEN );
        for ( int i = 0 ; i < labels.length ; i++ )
            guiFrame.getContentPane().add(labels[i]);

        but1 = new JButton("Press me"); // Create the new button
        guiFrame.getContentPane().add(but1);

        guiFrame.getContentPane().setLayout( new FlowLayout() );
        guiFrame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        guiFrame.setTitle("Hello World!");
        guiFrame.setLocationRelativeTo(null);
        guiFrame.pack();
        guiFrame.setVisible(true);
    }
}
```

The important details: a close-up

```
JFrame guiFrame = new JFrame();
ColorLabel labels[] = new ColorLabel[3];
JButton but1; // Initialised to null as a member

public void createGUI()
{
   labels[0] = new ColorLabel( 100, 100, Color.RED );
   labels[1] = new ColorLabel( 100, 100, Color.BLUE );
   labels[2] = new ColorLabel( 100, 100, Color.GREEN );
   for ( int i = 0 ; i < labels.length ; i++ )
      guiFrame.getContentPane().add(labels[i]);

   but1 = new JButton("Press me"); // Create new button
   guiFrame.getContentPane().add(but1);

   … set up the frame etc …
}
```

Previously, we wanted to add a button

- Create button(s) to add:

```
JButton but1 = new JButton("Hello");
```

- Tell it which object to notify:

```
but1.addActionListener( ActionListener needed );
```

- We had only one object so we used that one - tell button to notify the main object when it is clicked:

```
but1.addActionListener(this);
```

- Main class needed to **BE** An ActionListener (IS-A = Inheritance)

```
public class GUIHelloWorld implements ActionListener
```

- Then implemented the only method on the interface:

```
public void actionPerformed(ActionEvent e)
{…     }
```

We could use an **inner** class instead…

```
// Define this INSIDE the outer class!!!
public class ButtonPressed implements ActionListener
{
	@Override
	public void actionPerformed(ActionEvent e)
	{
		but1.setText( but1.getText()+".");
		guiFrame.pack();
	}
}
```

- New class implements ActionListener instead of the outer class
- Move actionPerformed method into it
- Create new instance for addActionListener:

```
but1.addActionListener(new ButtonPressed());
```

- Has access to members of the surrounding outer class – e.g. frame
- Can now create multiple buttons easily

We could add buttons with other actions

```
public class ButtonPressed2 implements ActionListener
{
	@Override
	public void actionPerformed(ActionEvent e)
	{
		but1.setText( "Button 2" );
		guiFrame.pack();
	}
}
```

- Create a new inner class to handle the other button - separately
- Create new instance for addActionListener:

```
JButton but2 = new JButton("Reset");
guiFrame.getContentPane().add(but2);
but2.addActionListener(new ButtonPressed2());
```

- Reference doesn't need to be a member, unless we need it later

A generic handler – check event source

```
public class ButtonPressed3 implements ActionListener
{
     @Override
     public void actionPerformed(ActionEvent e)
     {
          if ( e.getSource() instanceof JButton ) // Is it a button
          {
               System.out.println( "You pressed " +
                    ((JButton)(e.getSource())).getText() ); // cast
               if ( e.getSource() == but1 )
               {
                    but1.setText( but1.getText()+"." );
                    guiFrame.pack();
               }
               else
               {
                    but1.setText( "Cleared");
                    guiFrame.pack();
}    }    }    } // Close all of the braces – should be indented!
```

Other listeners work similarly

Not needed for coursework but useful to know

MouseListener Interface

```
public interface MouseListener extends EventListener
{
   /* Mouse button clicked (pressed and released) */
   public void mouseClicked(MouseEvent e);

   /* Mouse button has been pressed */
   public void mousePressed(MouseEvent e);

   /* Mouse button released */
   public void mouseReleased(MouseEvent e);

   /* Invoked when the mouse enters a component. */
   public void mouseEntered(MouseEvent e);

   /* Invoked when the mouse exits a component. */
   public void mouseExited(MouseEvent e);
}
```

Interface extends interface

Note:
EventListener has no methods

Using the interface : MyMouseHandler

```
class MyMouseHandler implements MouseListener
{
    public void mouseClicked(MouseEvent e) { … ? … }
    public void mousePressed(MouseEvent e) { … ? … }
    public void mouseReleased(MouseEvent e) { … ? … }
    public void mouseEntered(MouseEvent e) { … ? … }
    public void mouseExited(MouseEvent e) { … ? … }
}
```

```
// Create an object and listen with it…
label.addMouseListener( new MyMouseHandler() );
```

MouseMotionListener Interface

```
public interface MouseMotionListener extends EventListener
{
   /* Mouse moved when button pressed */
   public void mouseDragged(MouseEvent e);

   /* Mouse moved when button not pressed */
   public void mouseMoved(MouseEvent e);
}

// Create an object and listen with it…
label.addMouseMotionListener( new MouseMotionHandler() );
```

- This listens for motion (movement) of the mouse
- There are also many other similar interfaces and classes

Adapters

- Adaptors exist for many of the interfaces
- These are **classes** which have **empty** implementations of the methods, so you only implement the one that you want…
- **`MouseAdapter`** ***implements*** BOTH MouseListener and MouseMotionListener

- You can then create a class to **extend the adapter** (not implement it, as it is now a class) and just implement the methods you need

```
class MyMouseHandler extends MouseAdapter
{
    public void mouseClicked(MouseEvent e) {}
//    public void mousePressed(MouseEvent e) {}
//    public void mouseReleased(MouseEvent e) {}
//    public void mouseEntered(MouseEvent e) {}
//    public void mouseExited(MouseEvent e) {}
// Etc for the mouseMotion methods…
}

label.addMouseListener( new MyMouseHandler() );
label.addMouseMotionListener( new MyMouseHandler() );
```

Next Lecture : Lecture 11

- Boxing and autoboxing
- Generic classes (parametric polymorphism)
- Using the inner classes that we saw here
- (In Lecture 13 we will return to inner classes, to look at anonymous classes and Lambdas)